

AI Engineer: Take-Home Test (Document Extraction)

Overview

Build a **document extraction system** that pulls structured fields from real-world documents using an **LLM, VLM (vision-language model), or OCR + LLM** approach, and an **evaluation harness that proves the extractions are accurate**.

We are less interested in a demo that works on a clean PDF and more in a system you can *prove* is accurate on messy documents, that knows when it is unsure, and that you can reason about in terms of quality, latency, and cost. Treat evaluation as a first-class deliverable.

The Scenario

A client wants to automate extraction from a stack of documents (invoices, receipts, forms, contracts, IDs, your call). The documents are real: scanned, multi-page, inconsistently laid out. Some fields are simple lookups; others require reasoning across sections (a computed total, a party's role, line items in a table).

Pick any document type and dataset with real layout variation. Finding or building your own source counts in your favor, and your own scans count most. The dataset is a means to an end; we are evaluating the system around it.

What to Build

1. **Extraction.** Define a sensible field schema for your chosen documents and extract it with an LLM, a VLM, or OCR + LLM. The output should be typed and validated, the system should handle the messy parts (multi-page, tables, absent fields), and it must not hallucinate values that are not there. The system should also know when it is unsure: report per-field confidence and abstain rather than guess. The architecture is your call, agentic or a single well-designed extraction call; justify why your choice fits the problem.
2. **Evaluation.** This is what separates a strong submission. Build ground truth, pick metrics that are honest about how the system fails (not just how often), and make it runnable as a scorecard. Show how you would compare two versions and catch a regression before it ships.
3. **Production.** Show it handling more than one document at a time, and make cost and latency visible. Apply at least one optimization and report its impact.
4. **Agentic development.** Briefly describe how you used AI coding agents to build this, and ship one custom extension you actually used.

How you scope and shape each of these is up to you. We are looking for the judgment you bring, not adherence to a checklist.

Technical Notes

- **Language:** Python or TypeScript/Node. Any LLM/VLM provider; self-hosted or local is a bonus.
- **API access:** keep the provider configurable (any OpenAI-compatible endpoint is fine). We recommend a free option such as Google AI Studio (has vision/VLM), OpenRouter free models, or a local model via Ollama.
- Frameworks are fine for glue, but the extraction logic and eval harness should be your own and clearly understood. We may ask you to walk through the code and make a small live change.
- **Docker:** `docker compose up` should bring up the extraction API and leave it serving. The end-to-end evaluation should be a separate one-shot command in the same compose project (something like `docker compose run --rm eval`) that runs the harness against that API and writes the scorecard to a mounted output directory.
- Ship sample documents and ground truth in the repo so the eval runs out of the box, and commit the output artifacts from a run you actually did (the scorecard and the per-document results). Never commit real secrets; provide a `.env.example`.

Submission

- A **private repository** with the reviewer (provided in the email) invited. Commit everything, including dev artifacts; meaningful git history is a plus.
- A **README** covering: run commands, your dataset/schema/approach and why, architecture, key trade-offs, your eval results, the agentic-development note, and what you would do with more time.

Evaluation Criteria

Aspect	What We Look For
Evaluation Rigor	Ground truth you can trust; failures understood by kind, not just counted; results that stay comparable across versions
Extraction Quality	Accurate on messy, real documents; multi-page, tables, and cross-section fields handled well; no hallucination
LLM/VLM Engineering	A sound, defensible approach; reliable validated output; meaningful confidence and abstention
Production-Readiness	Cost/latency visibility; a real optimization with measured impact; resilience to bad output
Agentic Development	Effective, customized use of AI coding agents; a useful extension you actually built
Code Quality & Communication	Clean, well-structured code; clear README and trade-off reasoning